

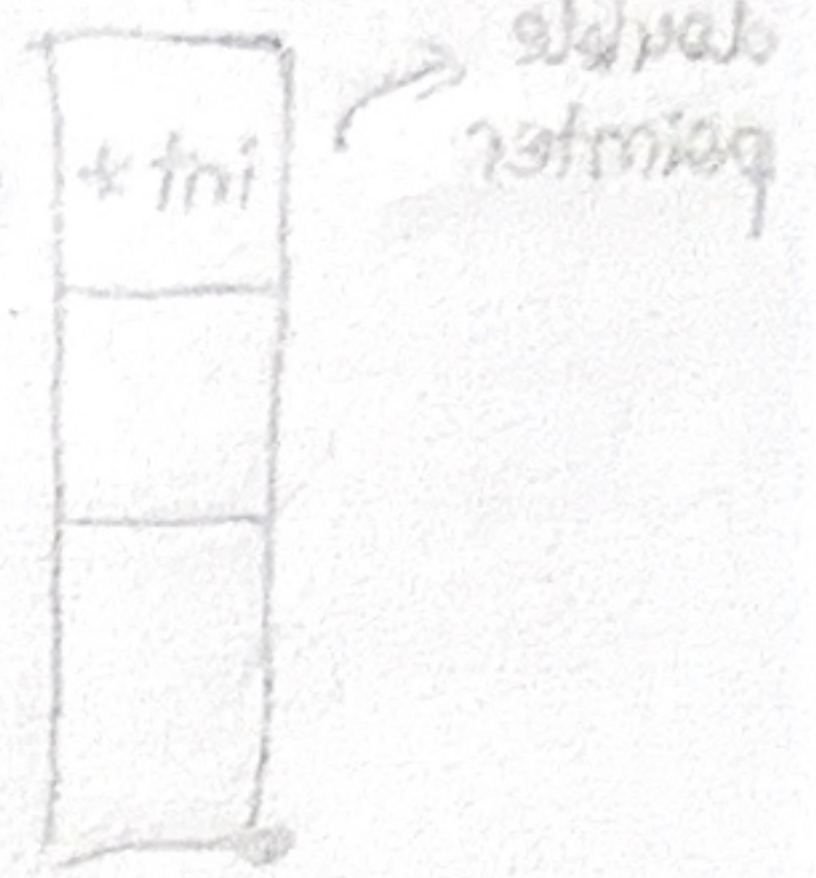
STACK

vs.

HEAP

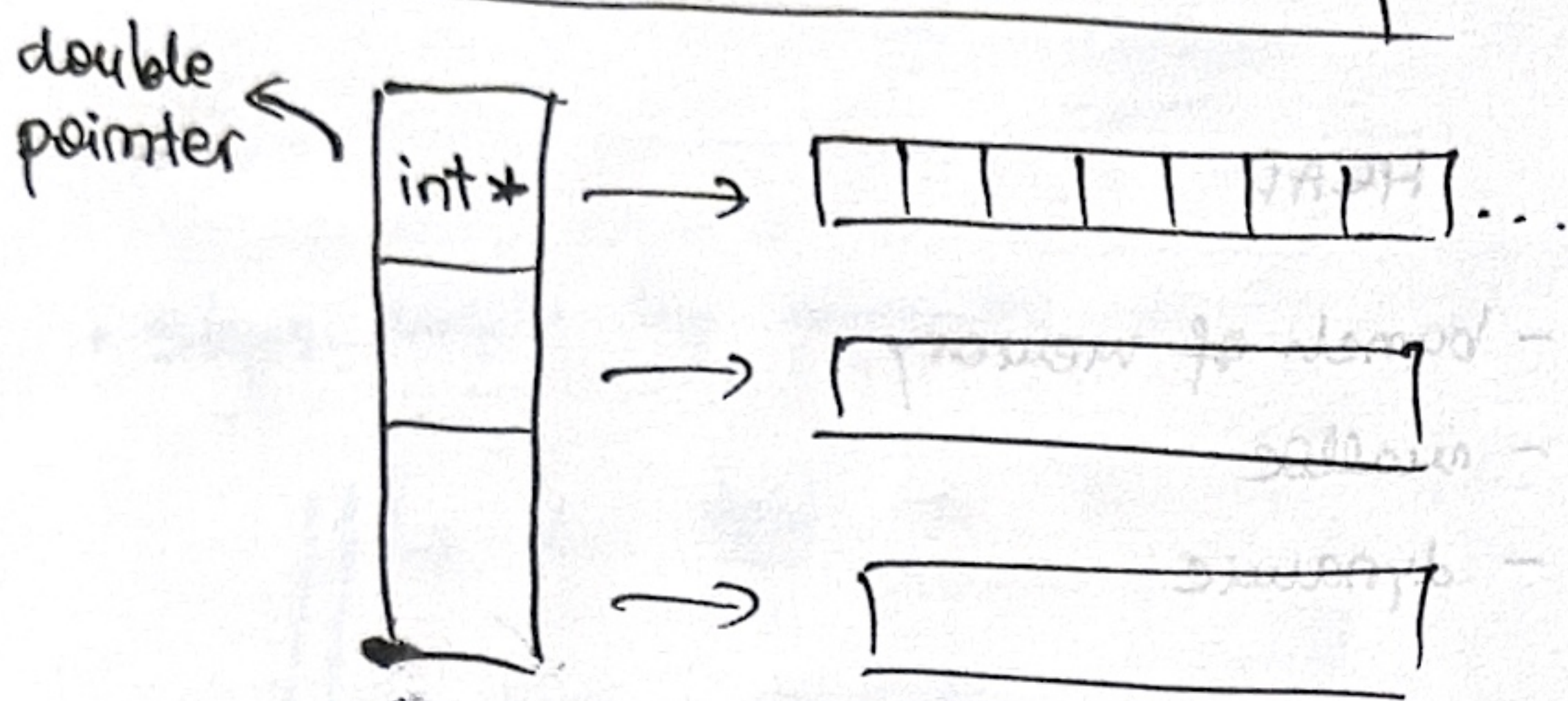
- area of memory
- fixed size
- limited memory

- bunch of memory
- malloc
- dynamic



	CODE	STACK	HEAP
STATIC ALLOCATION	<code>int m = 7;</code>		
	<code>int a[3];</code>		
	<code>int b[2][2];</code>		
DYNAMIC ALLOCATION	<code>int c[m]</code> <code>int m[3][4]</code>		<pre> m = (int**) malloc(3 * sizeof(int*)) for (i = 0; i < 3; i++) m[i] = (int*) malloc(4 * sizeof(int)) </pre> → m is a pointer to pointers → <code>malloc(3 * sizeof(int*))</code> allocates space for 3 pointers (rows) on the heap. → each <code>m[i] = (int*) malloc(4 * ...)</code> allocates 4 integers for that row. ⇒ 3x4 bidimensional matrix (3 rows of 4 ints each).
	<code>int *p = malloc(...)</code> (x64)	pointer variable (address) (the pointer points to memory in the heap, where the actual array elements live; when <code>free()</code> is called, the heap block is <u>marked as free</u>, but the "stack pointer still exists")	data → <u>"marked as free"</u>
	<code>int **m;</code> "pointer to pointer to int"	m → pointer to the first ptr. m[0] → pointer to int (first) m[1] → pointer to (second row) m[1][1] → element	

Bidimensional matrix in Heap.



cast: int *

double cast: `int **`

↳ array of pointers

int *p → POINTER TO INT.

→ when doing $p[i]$ $\Rightarrow$ array of integers

(because pointer arithmetic accesses consecutive ints)

int **m → POINTER TO POINTER TO INT

→ when doing `m[i]` $\Rightarrow$ array of pointers.

(ex: rows of a matrix)

$$\text{int} * p \rightarrow p[i] = \text{int}$$

$\text{int} **m \rightarrow m[i] = \text{int} *$

$$m[i][j] = i \cdot n + j$$

$\text{free}(u) \Rightarrow \text{free just}$

$\Rightarrow$ lose pointers to rows.

(heap block)

! 1) First free m[i]

2) Then free m.

1 Allocate + deallocate

valgrind $\rightarrow$ still some bytes in there that need to be cleaned.

3-dimensional matrix:

`int ***m` → "like a box"

`m` → pointer to int pointers

`m[i]` → pointer to *int

`m[i][j]` → pointer to **int

`m[i][j][k]` → actual int value.

! Java, Python → they deallocate automatically.

c++ / C → doesn't.

↳ "Andrei Alexandrescu" book.

Shell programming

Standard Input / Output

"everything is a file"

→ reading from command line (`scanf`)

→ `printf`, `cout`, `print` → "standard output"

↓
file descriptor

open
⇒ I get back an INT.

fopen ⇒ I get back a pointer (FILE)

- open
- read
- write
- close

} work with INT's.

StandardInput = 0

" - " - Output = 1

" - " - Error = 2

} can be redirected into a file.

`gets` → does not protect how much you read.

`p = fgets(s, 64, stdin)`

↓ ↳ standard input

64
bytes

→ reads at most $64 - 1 = 63$ bytes
because the last position is
reserved for the '\0'.

"at most one less than size".

String = array of bytes ending with 0.

LIBRARY FUNCTIONS

`fopen()`
`fprintf()`
`fgets()`
`fputs()`

- easier for text
- easier to use
- high-level
- from standard libraries

vs.

SYSTEM CALLS

`open()`
`read()`
`write()`
`close()`

- directly to OS kernel
- low-level operations

↓
files, processes ...

- faster
- no strings

Ex: Vowels from a file?

- ↳ character by character ⇒ slowest, seems "safest".
(head)
- ↳ best: read chunks.

`int main (int argc, char **argv)`

lets a program receive
command-line arguments.

`argc` = nr. of arguments

`argv` = array of strings (arguments)

ex: `ls -l` → inside main:

`argc = 2`

`argv[0] = "ls"`

`argv[1] = "-l"`

Stop program: `CTRL + C` = Kill

~Pause

: `CTRL + Z`.

OR

Read until end of file: `CTRL + D`.

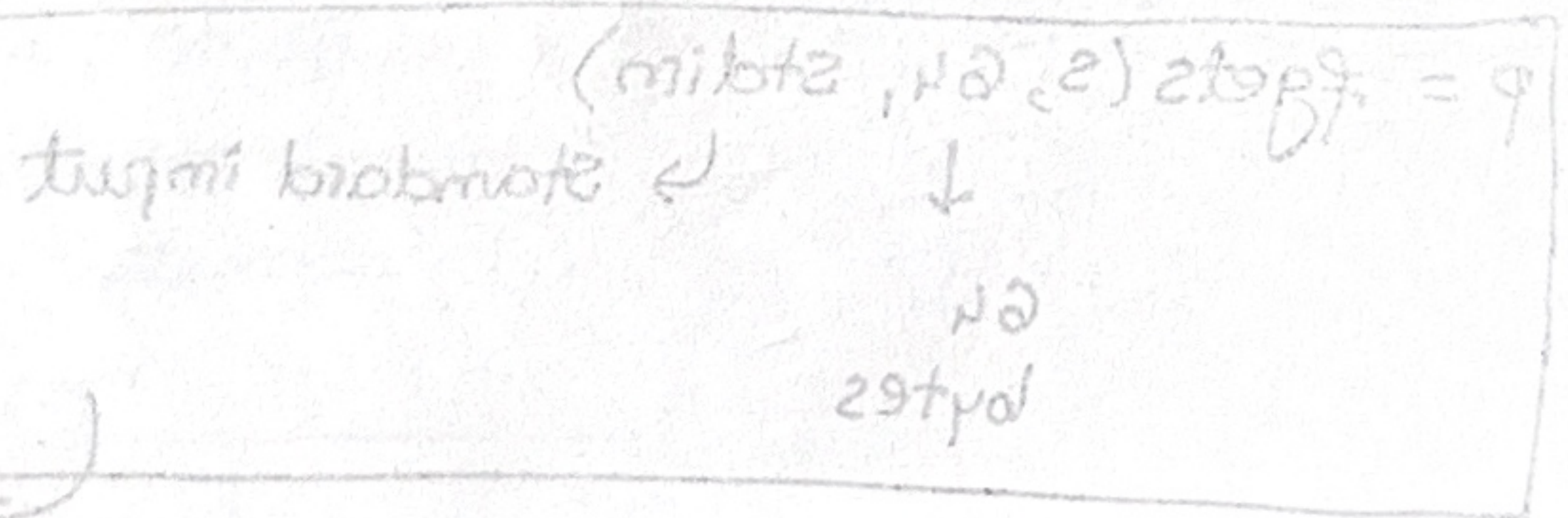
(Shell still waits for user input? ⇒ `CTRL + D`)

vi → start.c.swp.

if no diff tool with gcc

'o' not refs buffered

"9878" not read one to two to



Create vim "a.sh"

↳ #!/bin/bash → ?

ls

SCRIPT: → special var?

#!/bin/bash

echo \$0

echo \$1

echo \$9

echo \$*

echo \$@

echo \$#

shift 7 (left)

echo \$*

echo \$#

for A in x y z ; do

echo \$A

done

.txt 'who | awk '{print \$1}' '\$

⇒ terminal:

./a.sh a b c d e f g h i j

for A in \$* ; do

(if) test -f \$A; then

echo \$A is a file

elif test -d \$A; then

echo \$A is a directory

elif echo \$A | grep -E -q "[0-9]+\$"; then

echo \$A is a number

else

echo

(fi)

done

while read x; do

echo I read \$x

done

- > `ls > a.txt` "put content of ls to a.txt" → redirection.
- > `cat a.txt`
- > `ls > a.txt` "overwrites content".
- > `cat a.txt`
- > `ls >> a.txt` "append content of ls to end of a.txt".
- > `cat a.txt`
- > `./stars < a.txt` "read input from a.txt, not from keyboard".
- > `./stars lib < a.txt`
argument

- > `ls > b.txt` "redirect standard input, output".
- > `ls ..... > e.txt 2 > &1` "redirect stdout (1) to e.txt";
"2 > &1 redirects stderr (2) to wherever stdout goes".

⇒ both normal output and error message go to e.txt

- > `ls ..... > e.txt 2 > e.txt` "sends stdout → e.txt"
"2 > sends stderr → e.txt"

`>` = send output to file

`2>` = send errors to file

`2 > &1` = send errors to the same place as stdout (1).

`/dev/null` → ~~dis~~ device file that discards everything written to it.
→ anything sent to /dev/null is ignored (deleted).
→ used to suppress unwanted output or errors.

Truth value of a command

- every command = condition.

exit code = truth value 0 $\Rightarrow$ "successful"

$\neq 0 \Rightarrow$ error.

vs

c: 0 $\Rightarrow$ error

$\neq 0 \Rightarrow$ successful

test command

test -f a.txt

echo \$?

$\hookrightarrow 0$

(success)

test -f a.txt; echo \$?

echo \$?

$\hookrightarrow 1$

(error)

true || echo $\rightarrow$ prints nothing

false || echo $\rightarrow$ prints

28

Variables in shell:

A=5

B=star

echo \$A \$B $\Rightarrow$ 5 star.

echo \$A \${B}s $\Rightarrow$ 5 stars

echo \$A \$Bs $\Rightarrow$ 5.

du -s. $\Rightarrow$ 68 bytes

du -s . | awk '{print \$1}'

c='du -s . | awk '{print \$1}''

echo \$c

c=f

echo \$c

$\Rightarrow$ 1 rases rases

owner

group

(everybody)

chmod 700

base 2 $\Rightarrow$...